Neural Networks

1. Artificial Neuron

Suppose a program predicts whether a student will pass an examination.
We have two inputs:

- x_1: hours studied
- x_2: hours spent practising problems

Our output is:

- $y = 1$: likely to pass
- $y = 0$: likely to fail

Training data:

Student	**Study hours x_1**	**Practice hours x_2**	**Result y**
A	1	1	0
B	2	1	0
C	4	3	1
D	5	4	1

The task is to learn a relationship between the inputs and the output.

An **artificial neuron** is a mathematical operation with four main steps.
1. Inputs: Information given to the neuron, such as study hours and practice hours.
2. Weighted sum: $z = w_1x_1 + w_2x_2 + b$ Multiply inputs by weights and add a bias.
3. Activation function: $a = \phi(z)$ Transform the weighted sum into the neuron's output.
4. Output: A score, probability, or binary decision, depending on the model.

01_artificial_neuron.py

```
import math

# Inputs
x1 = 3.0
x2 = 2.0

# Weights and bias
w1 = 0.8
w2 = 0.5
b = -3.0

# Step 1: Calculate the weighted sum
z = w1 * x1 + w2 * x2 + b

# Step 2: Apply sigmoid activation
output = 1 / (1 + math.exp(-z))

# Step 3: Convert the output to a binary decision
prediction = PASS if output >= 0.5 else FAIL

print("Weighted sum:", z)
print("Neuron output:", output)
print("Prediction:", prediction)
```

```
PS C:\Users\Lenovo\ML Journey\Neural Networks> py 01_artificial_neuron.py
Weighted sum: 0.40000000000000036
Neuron output: 0.5986876601124521
Prediction: PASS
```

For given parameters: $w_1 = 0.8,\ w_2 = 0.5,\ b = -3$

For a student who studies for 3 hours and practices for 2 hours: $x_1 = 3,\ x_2 = 2$
The weighted sum is

$$\begin{aligned} z &= w_1 x_1 + w_2 x_2 + b \\ &= 0.8(3) + 0.5(2) - 3 \\ &= 2.4 + 1 - 3 \\ &= 0.4 \end{aligned}$$

For the activation function as sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$
For $z = 0.4$, $\sigma(0.4) \approx 0.599$
The neuron outputs approximately 0.599.
It corresponds to about a 59.9% predicted probability of passing. A classification threshold of 0.5 would classify this student as likely to pass.

2. Perceptron

A perceptron is one of the simplest neural-network models. It calculates a weighted sum and applies a threshold function.
Its activation can be written as $\hat{y} = \begin{cases} 1, & w^T x + b \geq 0 \\ 0, & w^T x + b < 0 \end{cases}$
Here, $w^T x$ means the sum of the products of corresponding weights and inputs.
Unlike sigmoid, this threshold activation directly outputs zero or one.

The decision boundary: For two input features, the boundary between the two classes is $w_1 x_1 + w_2 x_2 + b = 0$
Rearranging gives $x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}$
This equation represents a straight line.

A perceptron can learn a straight-line decision boundary for two features, or a hyperplane when there are more features.

3. Hidden Layer

Consider the XOR problem. Its output is 1 when the inputs differ and 0 when they are equal.

x_1	x_2	**XOR output**
0	0	0
0	1	1
1	0	1
1	1	0

The positive examples are $(0,1)$and $(1,0)$. The negative examples are $(0,0)$ and $(1,1)$.
A single perceptron cannot separate these four points with one straight line.
The solution is to combine multiple neurons into a network with a hidden layer.
The hidden neurons learn intermediate representations. The output neuron combines them to produce the final answer.
A simple network could implement XOR using these threshold functions:

$$h_1 = x_1 \wedge \neg x_2$$
$$h_2 = \neg x_1 \wedge x_2$$
$$y = h_1 \vee h_2$$

The hidden neurons detect the two cases where the inputs differ. The output neuron combines those cases.
This is a hand-designed example, but training a neural network can learn appropriate weights to achieve the same behaviour.

4. Multilayer perceptron (MLP)

An MLP is a feedforward neural network consisting of an input layer, one or more hidden layers, and an output layer.

i. The input layer receives the data
ii. Hidden layers transform the information
iii. The output layer produces the model's prediction.

For example, a network with 2 input features, 3 hidden neurons and 1 output neuron has:

- 2 input values.
- 3 hidden-layer weights for each input, plus 3 hidden biases.
- 3 output weights, plus 1 output bias.

The total number of trainable parameters is $(2 \times 3) + 3 + (3 \times 1) + 1 = 13$
Notice that the input features themselves are not trainable parameters but the weights and biases are.

5. Forward propagation

Forward propagation means passing the input through the network to calculate its prediction.
For a two-layer network:

$z^{[1]} = W^{[1]}x + b^{[1]}$ $a^{[1]} = \phi(z^{[1]})$ $z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$ $\hat{y} = \sigma(z^{[2]})$	W: weight matrix. b: bias vector. z: weighted sums. a: activated values. $\hat{y}$: predicted output.

(The superscripts identify the layer; they are not exponents.)

6. Activation functions

Sigmoid: $\sigma(z) = \frac{1}{1+e^{-z}}$
Output range: 0 to 1. Commonly used in a binary classifier's output layer. Its gradients can become very small for large positive or negative inputs.

Tanh: $\tanh(z)$
Output range: −1 to 1. It is zero-centred, but it can also suffer from vanishing gradients.

ReLU: $\mathrm{ReLU}(z) = \max(0, z)$
Negative inputs become zero; positive inputs pass through. It is widely used in hidden layers.

Softmax: $p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$
Converts multiple class scores into probabilities that sum to 1. Commonly used for mutually exclusive multiclass classification.

Leaky ReLU: $\mathrm{LReLU}(z) = \begin{cases} z & z \geq 0 \\ \alpha z & z < 0 \end{cases}$
Leaky ReLU retains a small negative slope. For example, if $\alpha = 0.01$, then $\mathrm{LReLU}(-3) = -0.03$

Other functions include Leaky ReLU, PReLU, ELU, SELU, GELU, Swish and sine activations.

7. Feature transformation
A feature transformation changes the representation of the input data. Suppose your original inputs are x_1 and x_2. Instead of feeding these features directly to a model, you could construct new features:

$$\phi(\mathbf{x}) = \begin{bmatrix} x_1 \\ x_2 \\ x_1 x_2 \end{bmatrix}$$

The new feature $x_1 x_2$ captures an interaction between the original inputs.
Other possible transformations include: $x_1^2,\ x_2^2,\ |x_1 - x_2|,\ \sin(x_1)$

Original features (x_1, x_2)

⬇

Feature transformation $\phi(\mathbf{x})$

⬇

Linear classifier in the transformed space $\mathbf{w}^T \phi(\mathbf{x}) + b$

Solving the XOR problem by transforming the features:
Introduce an interaction feature: $x_3 = x_1 x_2$
Then use the transformed representation $\phi(\mathbf{x}) = (x_1, x_2, x_1 x_2)$
A linear threshold classifier in this new space can represent XOR using $z = x_1 + x_2 - 2x_1 x_2 - 0.5$
The output is 1 if $z \geq 0$, otherwise 0.

x_1	x_2	$x_1 x_2$	z	**Prediction**
0	0	0	−0.5	0
0	1	0	0.5	1
1	0	0	0.5	1
1	1	1	−0.5	0

Here the classifier itself is not nonlinear, we introduced an additional feature that lets a linear threshold function express the desired decision.

02_xor_problem.py

```
# Lesson 2: Feature transformation and XOR

def step(z):
    """Return 1 if z >= 0, otherwise return 0."""
    return 1 if z >= 0 else 0


# Four XOR input examples
data = [
    (0, 0, 0),
    (0, 1, 1),
    (1, 0, 1),
    (1, 1, 0)
]

print("METHOD A: TRANSFORMED FEATURES")
print("x1 x2 x1*x2 z prediction")

for x1, x2, target in data:

    # Create a new feature
    interaction = x1 * x2

    # Linear classifier in transformed feature space
    z = x1 + x2 - 2 * interaction - 0.5

    # Threshold activation
    prediction = step(z)

    print(x1, x2, interaction, z, prediction)
```

```
print("\nMETHOD B: TWO HIDDEN NEURONS")
print("x1 x2 h1 h2 prediction")

for x1, x2, target in data:

    # Hidden-layer calculations
    h1 = step(x1 - x2 - 0.5)
    h2 = step(-x1 + x2 - 0.5)

    # Output-layer calculation
    prediction = step(h1 + h2 - 0.5)

    print(x1, x2, h1, h2, prediction)
```

```
PS C:\Users\Lenovo\ML Journey\Neural Networks> py 02_xor_problem.py
METHOD A: TRANSFORMED FEATURES
x1 x2 x1*x2 z prediction
0 0 0 -0.5 0
0 1 0 0.5 1
1 0 0 0.5 1
1 1 1 -0.5 0

METHOD B: TWO HIDDEN NEURONS
x1 x2 h1 h2 prediction
0 0 0 0 0
0 1 0 1 1
1 0 1 0 1
1 1 0 0 0
```

Method B — Use a neural network with a hidden layer

Instead of manually constructing the interaction feature, we can combine hidden neurons. Define two hidden units:

$$h_1 = \text{step}(x_1 - x_2 - 0.5)$$
$$h_2 = \text{step}(-x_1 + x_2 - 0.5)$$

Here, $\text{step}(z) = 1$ when $z \geq 0$, and 0 otherwise.

The first hidden neuron detects $(1,0)$. The second detects $(0,1)$.

The output neuron combines them: $y = \text{step}(h_1 + h_2 - 0.5)$

8. The Perceptron Learning Algorithm

Initially, we don't know the correct weights. We might start with $w_1 = 0,\ w_2 = 0,\ b = 0$

The perceptron calculates its prediction using $z = w_1 x_1 + w_2 x_2 + b$

Then it applies the threshold function: $\hat{y} = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases}$

If the prediction is wrong, we adjust the parameters. If it is correct, the basic perceptron learning algorithm makes no update for that example.

The perceptron learning rule is $\boxed{w_i^{\text{new}} = w_i^{\text{old}} + \eta(y - \hat{y})x_i \ = \ w_i^{\text{old}} + \eta e x_i}$

The bias update is $\boxed{b^{\text{new}} = b^{\text{old}} + \eta(y - \hat{y}) \ = \ b^{\text{old}} + \eta e}$

The learning rate controls the size of each update. For our first example, we'll choose \(\eta=1\) to keep the arithmetic simple.

Example: learning the AND gate

The AND gate outputs 1 only when both inputs are 1.

x_1	x_2	**Target** y
0	0	0
0	1	0
1	0	0
1	1	1

We'll use: $w_1 = 0,\ w_2 = 0,\ b = 0,\ \eta = 1$

Our threshold rule is $z \geq 0 \Rightarrow \hat{y} = 1$. Therefore, when all parameters start at zero, the first prediction is 1.

Step 1: Process input $(0,0)$

The correct target is $y = 0$.

Calculate the weighted sum: $z = 0(0) + 0(0) + 0 = 0$

Because $z \geq 0$, the prediction is $\hat{y} = 1$

The error is $e = y - \hat{y} = 0 - 1 = -1$

Update the parameters:

$$w_1 = 0 + 1(-1)(0) = 0$$
$$w_2 = 0 + 1(-1)(0) = 0$$
$$b = 0 + 1(-1) = -1$$

Our new parameters are $\boxed{w_1 = 0, \quad w_2 = 0, \quad b = -1}$

The weights remain unchanged because both inputs are zero, but the bias changes.

Step 2: Process input $(0,1)$

The target is $y = 0$.

Using the updated parameters: $z = 0(0) + 0(1) - 1 = -1$

Since$z < 0$, $\hat{y} = 0$

This is correct, so $e = 0$. No parameters change.

Step 3: Process input $(1,0)$

Similarly, $z = 0(1) + 0(0) - 1 = -1$

The prediction is 0, which is correct. No update occurs.

Step 4: Process input $(1,1)$

The target is $y = 1, and\ z = 0(1) + 0(1) - 1 = -1$, therefore the prediction is 0, so the error is $e = 1 - 0 = 1$

Update both weights and the bias:

$$w_1 = 0 + 1(1)(1) = 1$$
$$w_2 = 0 + 1(1)(1) = 1$$
$$b = -1 + 1(1) = 0$$

After the first epoch, our parameters are $\boxed{w_1 = 1, \quad w_2 = 1, \quad b = 0}$

However with the new parameters, the input $(0,0)$ again produces $z = 0$, so it is incorrectly classified as 1. We update the bias to -1.

Next, $(0,1)$produces $z = 0$ and is incorrectly classified as 1. Updating gives $w_2 = 0$ and $b = -2$.

The next examples continue adjusting the parameters. Repeated passes through the dataset eventually produce a set of weights and bias that classify all four AND examples correctly.

03_perceptron_learning_algorithm.py

```
# Lesson 3: Perceptron learning algorithm
# Example: Learn the AND logic gate

# Step 1: Create the training dataset
data = [
    (0, 0, 0),
    (0, 1, 0),
```

```
    (1, 0, 0),
    (1, 1, 1)
]

# Step 2: Initialize parameters
w1 = 0.0
w2 = 0.0
b = 0.0

# Step 3: Set learning parameters
learning_rate = 1.0
max_epochs = 20

# Step 4: Define the activation function
def step(z):
    return 1 if z >= 0 else 0

# Step 5: Train the perceptron
for epoch in range(max_epochs):

    mistakes = 0

    for x1, x2, target in data:

        # Forward propagation
        z = w1 * x1 + w2 * x2 + b
        prediction = step(z)

        # Calculate prediction error
        error = target - prediction

        # Update parameters if prediction is wrong
        w1 = w1 + learning_rate * error * x1
        w2 = w2 + learning_rate * error * x2
        b = b + learning_rate * error

        if error != 0:
            mistakes += 1

    print(
        f"Epoch {epoch + 1}: "
        f"mistakes={mistakes}, "
        f"w1={w1:.1f}, w2={w2:.1f}, b={b:.1f}"
    )

    # Stop when every training example is classified correctly
    if mistakes == 0:
        break

# Step 6: Test the trained model
print("\nFinal predictions:")

for x1, x2, target in data:
    z = w1 * x1 + w2 * x2 + b
    prediction = step(z)

    print(
        f"Input=({x1}, {x2}), "
        f"Target={target}, Prediction={prediction}"
    )
```

```
PS C:\Users\Lenovo\ML Journey\Neural Networks> py 03_perceptron_learning_algorithm.py
Epoch 1: mistakes=2, w1=1.0, w2=1.0, b=0.0
Epoch 2: mistakes=3, w1=2.0, w2=1.0, b=-1.0
Epoch 3: mistakes=3, w1=2.0, w2=1.0, b=-2.0
Epoch 4: mistakes=2, w1=2.0, w2=2.0, b=-2.0
Epoch 5: mistakes=1, w1=2.0, w2=1.0, b=-3.0
Epoch 6: mistakes=0, w1=2.0, w2=1.0, b=-3.0

Final predictions:
Input=(0, 0), Target=0, Prediction=0
Input=(0, 1), Target=0, Prediction=0
Input=(1, 0), Target=0, Prediction=0
Input=(1, 1), Target=1, Prediction=1
```

04_forward_propagation.py

```
import numpy as np

# ========================================
# STEP 1: Define the input features
# ========================================

x = np.array([
    [1.0],
    [2.0]
])

# ========================================
# STEP 2: Define hidden-layer parameters
# ========================================

W1 = np.array([
    [0.5, -1.0],
    [1.0,  0.5]
])

b1 = np.array([
    [0.0],
    [-0.5]
])

# ========================================
# STEP 3: Define output-layer parameters
# ========================================

W2 = np.array([
    [1.0, -1.0]
])

b2 = np.array([
    [0.5]
])

# ========================================
# STEP 4: Define activation functions
# ========================================

def relu(z):
    return np.maximum(0, z)
```

```
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

# ========================================
# STEP 5: Forward propagation
# ========================================

# Hidden layer: weighted sum
z1 = W1 @ x + b1

# Hidden layer: activation
h = relu(z1)

# Output layer: weighted sum
z2 = W2 @ h + b2

# Output layer: sigmoid activation
y_hat = sigmoid(z2)

# ========================================
# STEP 6: Print intermediate calculations
# ========================================

print("Input x:")
print(x)

print("\nHidden weighted sum z1:")
print(z1)

print("\nHidden activation h:")
print(h)

print("\nOutput weighted sum z2:")
print(z2)

print("\nFinal output y_hat:")
print(y_hat)

print("\nPredicted class:")
print(int(y_hat[0, 0] >= 0.5))
```

```
PS C:\Users\Lenovo\ML Journey\Neural Networks> py 04_forward_propagation.py
Input x:
[[1.]
 [2.]]

Hidden weighted sum z1:
[[-1.5]
 [ 1.5]]

Hidden activation h:
[[0. ]
 [1.5]]

Output weighted sum z2:
[[-1.]]

Final output y_hat:
[[0.26894142]]

Predicted class:
0
```

05_linear_vs_nonlinear.py

```
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Generate input values
x = np.linspace(-1, 3, 200)

# Step 2: Network A - two linear layers
hidden_linear = 2 * x + 1
output_linear = 3 * hidden_linear - 2

# Step 3: Network B - a ReLU hidden layer
hidden_relu = np.maximum(0, 2 * x - 2)
output_relu = 3 * hidden_relu + 1

# Step 4: Print a few numerical comparisons
print("x | Linear output | ReLU output")

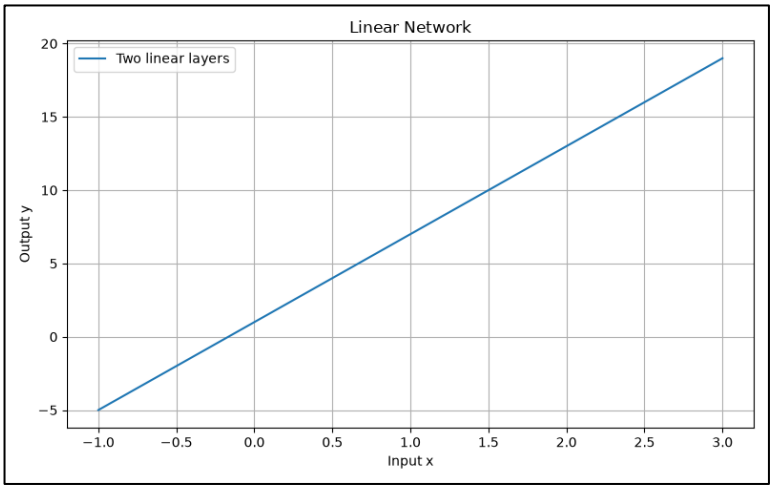
for value in [0, 0.5, 1, 1.5, 2, 3]:
    linear_y = 6 * value + 1
    relu_y = 3 * max(0, 2 * value - 2) + 1

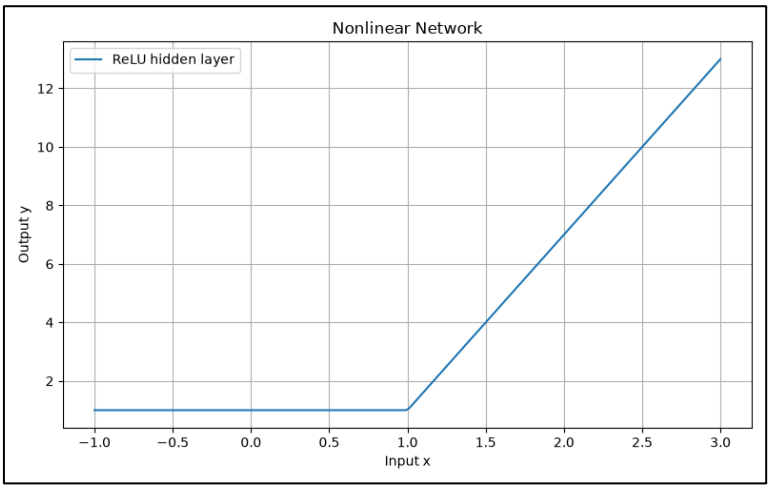
    print(f"{value:3.1f} | {linear_y:13.2f} | {relu_y:11.2f}")

# Step 5: Plot the linear network
plt.figure(figsize=(8, 5))
plt.plot(x, output_linear, label="Two linear layers")
plt.xlabel("Input x")
plt.ylabel("Output y")
plt.title("Linear Network")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()

# Step 6: Plot the nonlinear network
plt.figure(figsize=(8, 5))
plt.plot(x, output_relu, label="ReLU hidden layer")
plt.xlabel("Input x")
plt.ylabel("Output y")
plt.title("Nonlinear Network")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\Neural Networks> py 05_linear_vs_nonlinear.py
x | Linear output | ReLU output
0.0 |          1.00 |        1.00
0.5 |          4.00 |        1.00
1.0 |          7.00 |        1.00
1.5 |         10.00 |        4.00
2.0 |         13.00 |        7.00
3.0 |         19.00 |       13.00
```





9. Universal function approximation

The universal approximation theorem says, in one common form, that a feedforward neural network with a suitable nonlinear activation and a sufficiently large hidden layer can approximate any continuous function on a compact domain to arbitrary accuracy.

With the right architecture and enough hidden units, a neural network can approximate a very broad range of continuous input-output relationships.

Let the true function be $f(x)$, and the neural network prediction be $\hat{f}(x)$.

We want their difference to be small: $\left|f(x) - \hat{f}(x)\right| < \epsilon$ (error tolerance)

For example, if $\epsilon = 0.01$, we aim for an absolute error smaller than 0.01 throughout the specified domain.

The theorem tells us that an appropriate network can achieve such accuracy for the functions and domains covered by its assumptions.

What the theorem does not guarantee

- It does not say that every network architecture can approximate every function.
- It does not guarantee that training will discover the right weights.
- It does not guarantee good predictions on unseen data.
- It does not mean that a small network will approximate every function accurately.
- It does not guarantee that the required network will be computationally efficient.

Expressive capacity and successful training are different things. A model may be capable of representing a function but still fail to learn it from limited data or an unsuitable training procedure.

10. Loss function

A loss function measures the discrepancy between a model's prediction and the correct target. A loss function converts these errors into a numerical measure of prediction quality. A smaller loss generally indicates better predictions under that loss function. However, loss functions differ in how they penalize errors. That's why choosing the appropriate loss for the task matters.

Three important types of prediction tasks

1. Regression: Predict a continuous number, such as temperature, delay, or power consumption.
   Typical loss: Mean squared error.
2. Binary classification: Choose between two classes, such as pass/fail or defective/non-defective.
   Typical loss: Binary cross-entropy.
3. Multiclass classification: Choose one class from several mutually exclusive classes, such as identifying a defect type.
   Typical loss: Categorical cross-entropy.

A. **Mean squared error (MSE):** Mean squared error is commonly used for regression. It calculates the squared difference between the actual value and the predicted value, then averages the differences across the dataset.

The formula is $\boxed{\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$

Here:

- N is the number of samples.
- y_i is the actual target for sample \(i\).
- $\hat{y}_i$ is the predicted value.

The square makes every contribution nonnegative and penalizes large errors more heavily.

B. **Binary cross-entropy (BCE)**: Now suppose the task is not to predict a temperature but to predict whether a device is defective. The target is binary: $y \in \{0,1\}$. The neural network produces a probability-like prediction: $p = P(y = 1 \mid x)$. Binary cross-entropy measures how well the predicted probability matches the actual binary label. Its formula for one sample is $\boxed{L = -[y \log(p) + (1 - y)\log(1 - p)]}$. Here, log is the natural logarithm.

i. Case A: actual label is 1. Set $y = 1$: $L = -[1 \log(p) + 0 \log(1 - p)]$. Therefore, $\boxed{L = -\log(p)}$. If the actual class is positive, the model should assign a high probability to that class.

ii. Case B: actual label is 0. Set $y = 0$: $L = -[0 \log(p) + 1 \log(1 - p)]$. Therefore, $\boxed{L = -\log(1 - p)}$. If the actual class is negative, the model should assign a low probability to the positive class.

BCE across multiple samples: For N samples, average the individual losses:

$$\boxed{L_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(p_i) + (1 - y_i)\log(1 - p_i)]}$$

C. **Categorical cross-entropy (CCE)**: For one example with one-hot encoded target values, $\boxed{L_{\text{CCE}} = -\sum_{k=1}^{K} y_k \log(p_k)}$, where:

- K is the number of classes.
- $y_k = 1$ for the correct class and 0 for the other classes.
- p_k is the predicted probability of class \(k\).

06_loss_functions.py

```
import numpy as np

# ========================================
# PART 1: MEAN SQUARED ERROR
# ========================================

# Actual continuous values
y_true_reg = np.array([10.0, 20.0, 30.0])

# Predicted continuous values
y_pred_reg = np.array([12.0, 18.0, 33.0])

# Calculate squared errors
squared_errors = (y_true_reg - y_pred_reg) ** 2
```

```
# Average the squared errors
mse = np.mean(squared_errors)

print("MSE EXAMPLE")
print("Squared errors:", squared_errors)
print("MSE:", mse)


# ========================================
# PART 2: BINARY CROSS-ENTROPY
# ========================================

# Actual binary labels
y_true_bin = np.array([1.0, 0.0, 1.0, 0.0])

# Predicted probabilities for class 1
y_pred_bin = np.array([0.8, 0.2, 0.6, 0.1])

# Keep probabilities away from exactly 0 and 1
epsilon = 1e-15

p = np.clip(y_pred_bin, epsilon, 1 - epsilon)

# Calculate BCE for every example
bce_per_sample = -(
    y_true_bin * np.log(p)
    + (1 - y_true_bin) * np.log(1 - p)
)

# Average the losses
bce = np.mean(bce_per_sample)

print("\nBINARY CROSS-ENTROPY EXAMPLE")
print("Loss per sample:", bce_per_sample)
print("BCE:", bce)


# ========================================
# PART 3: CATEGORICAL CROSS-ENTROPY
# ========================================

# Three samples, three possible classes
y_true_cat = np.array([
    [1, 0, 0],
    [0, 1, 0],
    [0, 0, 1]
])

# Predicted class probabilities
y_pred_cat = np.array([
    [0.7, 0.2, 0.1],
    [0.1, 0.8, 0.1],
    [0.2, 0.2, 0.6]
])

# Avoid log(0)
p_cat = np.clip(y_pred_cat, epsilon, 1.0)

# Sum over classes for each sample
cce_per_sample = -np.sum(
```

```
    y_true_cat * np.log(p_cat),
    axis=1
)

# Average over samples
cce = np.mean(cce_per_sample)

print("\nCATEGORICAL CROSS-ENTROPY EXAMPLE")
print("Loss per sample:", cce_per_sample)
print("CCE:", cce)
```

```
PS C:\Users\Lenovo\ML Journey\Neural Networks> py 06_loss_functions.py
MSE EXAMPLE
Squared errors: [4. 4. 9.]
MSE: 5.666666666666667

BINARY CROSS-ENTROPY EXAMPLE
Loss per sample: [0.22314355 0.22314355 0.51082562 0.10536052]
BCE: 0.2656183105130591

CATEGORICAL CROSS-ENTROPY EXAMPLE
Loss per sample: [0.35667494 0.22314355 0.51082562]
CCE: 0.3635480396729776
```

11. The chain rule

In a neural network, a weight usually does not affect the loss directly in one simple step. It affects a neuron's weighted sum, which affects its activation, which affects the output, which affects the loss. The chain rule lets us calculate how the final loss changes through this chain of operations.

Suppose: $z = 2w, a = z^2,\ L = 3a$

We want to calculate $\frac{dL}{dw}$. The dependency chain is $w \longrightarrow z \longrightarrow a \longrightarrow L$. The chain rule says $\boxed{\frac{dL}{dw} = \frac{dL}{da}\frac{da}{dz}\frac{dz}{dw}}$.

Therefore, $\boxed{\frac{dL}{dw} = 24w}$.

A neural-network numerical example: sigmoid and MSE

Let's use a single neuron with one input. Suppose $x = 2,\ w = 0.5,\ b = 0$. The target is $y = 1$.

i. **Step 1 — Forward propagation:** Calculate the weighted sum: $z = wx + b = 0.5(2) = 1$. Apply sigmoid: $\hat{y} = \sigma(1) = \frac{1}{1+e^{-1}}$. Therefore, $\hat{y} \approx 0.7311$. The prediction is approximately 0.7311, while the correct target is 1.

ii. **Step 2 — Calculate the loss:** For this example, use the squared error for one sample:

$$L = (y - \hat{y})^2$$
$$L = (1 - 0.7311)^2$$
$$\boxed{L \approx 0.0723}$$

iii. **Step 3 — Calculate the gradient:** The loss depends on the weight through the sigmoid output. We use the chain rule: $\frac{dL}{dw} = \frac{dL}{d\hat{y}}\frac{d\hat{y}}{dz}\frac{dz}{dw}$. The first derivative of $L = (y - \hat{y})^2\ is \frac{dL}{d\hat{y}} = 2(\hat{y} - y)$. For sigmoid, $\frac{d\hat{y}}{dz} = \hat{y}(1 - \hat{y}), and$ because $z = wx + b, \frac{dz}{dw} = x$. Combining the three derivatives:

$$\boxed{\frac{dL}{dw} = 2(\hat{y} - y)\hat{y}(1 - \hat{y})x}$$

Substitute our values:

$$\frac{dL}{dw} = 2(0.7311 - 1)(0.7311)(1 - 0.7311)(2)$$
$$\boxed{\frac{dL}{dw} \approx -0.1058}$$

The negative gradient means that a small increase in the weight will locally decrease the loss.

With learning rate $\eta = 0.1$,

$$w_{\text{new}} = w - \eta \frac{dL}{dw}$$

$$w_{\text{new}} = 0.5 - 0.1(-0.1058)$$

$$\boxed{w_{\text{new}} \approx 0.5106}$$

The weight increases slightly, moving the prediction in the direction of the positive target.
This example uses squared error with a sigmoid output. Binary cross-entropy is another common choice for binary classification, and it produces a different loss derivative.

07_gradient_calculation.py

```
import numpy as np

# ========================================
# STEP 1: Define the training example
# ========================================

x = 2.0
target = 1.0

# Initial parameters
w = 0.5
b = 0.0

# Learning rate
learning_rate = 0.1


# ========================================
# STEP 2: Define sigmoid activation
# ========================================

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))


# ========================================
# STEP 3: Forward propagation
# ========================================

z = w * x + b
prediction = sigmoid(z)

# Squared error for this one sample
loss = (target - prediction) ** 2


# ========================================
# STEP 4: Apply the chain rule
# ========================================

dL_dprediction = 2 * (prediction - target)

dprediction_dz = prediction * (1 - prediction)

dz_dw = x

# Multiply the local derivatives
dL_dw = (
```

```
    dL_dprediction
    * dprediction_dz
    * dz_dw
)


# ========================================
# STEP 5: Calculate a numerical gradient
# ========================================

epsilon = 1e-5

def calculate_loss(weight):
    z_value = weight * x + b
    predicted_value = sigmoid(z_value)
    return (target - predicted_value) ** 2

loss_plus = calculate_loss(w + epsilon)
loss_minus = calculate_loss(w - epsilon)

numerical_gradient = (
    loss_plus - loss_minus
) / (2 * epsilon)


# ========================================
# STEP 6: Update the weight
# ========================================

new_w = w - learning_rate * dL_dw


# ========================================
# STEP 7: Display results
# ========================================

print("Weighted sum:", z)
print("Prediction:", prediction)
print("Loss:", loss)

print("dL/dprediction:", dL_dprediction)
print("dprediction/dz:", dprediction_dz)
print("dz/dw:", dz_dw)

print("Analytical gradient:", dL_dw)
print("Numerical gradient:", numerical_gradient)

print("Old weight:", w)
print("Updated weight:", new_w)
```

```
PS C:\Users\Lenovo\ML Journey\Neural Networks> py 07_gradient_calculation.py
Weighted sum: 1.0
Prediction: 0.7310585786300049
Loss: 0.07232948812851325
dL/dprediction: -0.5378828427399902
dprediction/dz: 0.19661193324148185
dz/dw: 2.0
Analytical gradient: -0.21150837113706686
Numerical gradient: -0.2115083711465293
Old weight: 0.5
Updated weight: 0.5211508371137067
```

12. Backpropagation

We know that, A gradient tells us how much the loss changes when a parameter changes slightly. For a parameter θ, the gradient is: $\frac{\partial L}{\partial \theta}$. A negative gradient means increasing that parameter slightly would reduce the loss locally; a positive gradient means decreasing it would reduce the loss locally.

Now, backpropagation calculates these gradients starting at the output and moving toward the input.

For a sigmoid output combined with binary cross-entropy, the derivative simplifies to: $\boxed{\delta_2 = \frac{\partial L}{\partial z_2} = \hat{y} - y}$.

for output weighted sum is: $z_2 = w_{21}a_{11} + w_{22}a_{12} + b_2$. Using the chain rule: $\frac{\partial L}{\partial w_{2j}} = \frac{\partial L}{\partial z_2}\frac{\partial z_2}{\partial w_{2j}} = \delta_2 a_{1j}$

Therefore, $\frac{\partial L}{\partial W_2} = \delta_2 a_1^T$

A hidden neuron does not directly produce the final prediction. Its influence passes through the output weights. Therefore, its error signal depends on both the output error and the output weight connected to it.

For hidden neuron j: $\boxed{\delta_{1j} = \frac{\partial L}{\partial a_{1j}} \text{ReLU}'(z_{1j})}$, where $\text{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases}$

First, propagate the output error through the output weights: $\frac{\partial L}{\partial a_1} = W_2^T \delta_2$

Recall: $z_1 = W_1 x + b_1$. The hidden-layer weight gradient is: $\boxed{\frac{\partial L}{\partial W_1} = \delta_1 x^T}$

The update rule is: $\boxed{\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_\theta L}$

08_backpropagation.py

```
import numpy as np

# ====================================================
# 1. INPUT AND TARGET
# ====================================================

x = np.array([[1.0],
              [2.0]])

y = np.array([[1.0]])

# ====================================================
# 2. INITIAL WEIGHTS AND BIASES
# ====================================================

W1 = np.array([
    [0.1, 0.2],
    [0.3, 0.4]
])

b1 = np.array([
    [0.1],
    [0.1]
])

W2 = np.array([[0.5, -0.4]])

b2 = np.array([[0.2]])

learning_rate = 0.1


# ====================================================
# 3. ACTIVATION AND LOSS FUNCTIONS
# ====================================================

def relu(z):
    return np.maximum(0, z)
```

```
def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))


def binary_cross_entropy(y, y_hat):
    eps = 1e-12
    y_hat = np.clip(y_hat, eps, 1 - eps)

    return -np.mean(
        y * np.log(y_hat)
        + (1 - y) * np.log(1 - y_hat)
    )


# ============================================================
# 4. FORWARD PASS
# ============================================================

z1 = W1 @ x + b1
a1 = relu(z1)

z2 = W2 @ a1 + b2
y_hat = sigmoid(z2)

loss_before = binary_cross_entropy(y, y_hat)

print("FORWARD PASS")
print("Hidden pre-activations:\n", z1)
print("Hidden activations:\n", a1)
print("Output pre-activation:\n", z2)
print("Predicted probability:\n", y_hat)
print("Loss before update:", loss_before)


# ============================================================
# 5. BACKWARD PASS
# ============================================================

# Sigmoid output + binary cross-entropy
dz2 = y_hat - y

# Output-layer gradients
dW2 = dz2 @ a1.T
db2 = dz2.copy()

# Propagate the gradient into the hidden layer
da1 = W2.T @ dz2

# Derivative of ReLU
dz1 = da1 * (z1 > 0)

# Hidden-layer gradients
dW1 = dz1 @ x.T
db1 = dz1.copy()

print("\nBACKWARD PASS")
print("Output error dz2:\n", dz2)
print("Output weight gradient dW2:\n", dW2)
print("Output bias gradient db2:\n", db2)
print("Hidden error dz1:\n", dz1)
print("Hidden weight gradient dW1:\n", dW1)
print("Hidden bias gradient db1:\n", db1)


# ============================================================
# 6. UPDATE PARAMETERS
# ============================================================
```

```
W1 -= learning_rate * dW1
b1 -= learning_rate * db1

W2 -= learning_rate * dW2
b2 -= learning_rate * db2


# ============================================================
# 7. FORWARD PASS AFTER THE UPDATE
# ============================================================

z1_new = W1 @ x + b1
a1_new = relu(z1_new)

z2_new = W2 @ a1_new + b2
y_hat_new = sigmoid(z2_new)

loss_after = binary_cross_entropy(y, y_hat_new)

print("\nAFTER ONE GRADIENT DESCENT UPDATE")
print("Updated W1:\n", W1)
print("Updated b1:\n", b1)
print("Updated W2:\n", W2)
print("Updated b2:\n", b2)
print("New predicted probability:", y_hat_new.item())
print("New loss:", loss_after)
print("Loss decreased:", loss_after < loss_before)
```

```
PS C:\Users\Lenovo\ML Journey\Neural Networks> py 08_backpropagation.py
FORWARD PASS
Hidden pre-activations:
 [[0.6]
 [1.2]]
Hidden activations:
 [[0.6]
 [1.2]]
Output pre-activation:
 [[0.02]]
Predicted probability:
 [[0.50499983]]
Loss before update: 0.6831971797266342

BACKWARD PASS
Output error dz2:
 [[-0.49500017]]
Output weight gradient dW2:
 [[-0.2970001 -0.5940002]]
Output bias gradient db2:
 [[-0.49500017]]
Hidden error dz1:
 [[-0.24750008]
 [ 0.19800007]]
Hidden weight gradient dW1:
 [[-0.24750008 -0.49500017]
 [ 0.19800007  0.39600013]]
Hidden bias gradient db1:
 [[-0.24750008]
 [ 0.19800007]]
```

```
AFTER ONE GRADIENT DESCENT UPDATE
Updated W1:
 [[0.12475001 0.24950002]
 [0.28019999 0.36039999]]
Updated b1:
 [[0.12475001]
 [0.08019999]]
Updated W2:
 [[ 0.52970001 -0.34059998]]
Updated b2:
 [[0.24950002]]
New predicted probability: 0.5689880989672954
New loss: 0.563895760773496
Loss decreased: True
```

For a hidden ReLU layer and a sigmoid output trained with binary cross-entropy:

Quantity	**Formula**
Output error	$\delta_2 = \hat{y} - y$
Output weight gradient	$\nabla_{W_2} L = \delta_2 a_1^T$
Hidden error	$\delta_1 = (W_2^T \delta_2) \odot \mathrm{ReLU}'(z_1)$
Hidden weight gradient	$\nabla_{W_1} L = \delta_1 x^T$

The bias gradients are the corresponding error signals: $\nabla_{b_2} L = \delta_2,\ \nabla_{b_1} L = \delta_1$
Here, $\odot$ means element-by-element multiplication.

13. Training

Training a neural network to learn XOR

Input x_1	Input x_2	Target y
0	0	0
0	1	1
1	0	1
1	1	0

09_training_loop.py

```
import numpy as np

# 1. XOR dataset
X = np.array([
    [0.0, 0.0],
    [0.0, 1.0],
    [1.0, 0.0],
    [1.0, 1.0]
])

y = np.array([
    [0.0],
    [1.0],
    [1.0],
    [0.0]
])

# 2. Fixed random seed for reproducibility
rng = np.random.default_rng(42)

# 3. Network architecture: 2 inputs, 4 hidden neurons, 1 output
W1 = rng.normal(0, 0.5, (2, 4))
```

```
b1 = np.zeros((1, 4))

W2 = rng.normal(0, 0.5, (4, 1))
b2 = np.zeros((1, 1))


def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-np.clip(z, -500, 500)))


def tanh_derivative(z):
    return 1.0 - np.tanh(z) ** 2


def binary_cross_entropy(y, p):
    p = np.clip(p, 1e-12, 1 - 1e-12)
    return -np.mean(y * np.log(p) + (1-y) * np.log(1-p))


def forward(X_batch):
    z1 = X_batch @ W1 + b1
    a1 = np.tanh(z1)

    z2 = a1 @ W2 + b2
    p = sigmoid(z2)

    return p, (z1, a1, p)


# 4. Training configuration
learning_rate = 0.1
epochs = 10000
batch_size = 4
n = len(X)

loss_history = []

# 5. Training loop
for epoch in range(epochs):

    # Shuffle examples at the start of each epoch
    order = rng.permutation(n)
    X_shuffled = X[order]
    y_shuffled = y[order]

    for start in range(0, n, batch_size):
        X_batch = X_shuffled[start:start + batch_size]
        y_batch = y_shuffled[start:start + batch_size]
        m = len(X_batch)

        # Forward pass
        p, (z1, a1, p) = forward(X_batch)

        # Backward pass
        dz2 = (p - y_batch) / m
        dW2 = a1.T @ dz2
        db2 = np.sum(dz2, axis=0, keepdims=True)

        da1 = dz2 @ W2.T
        dz1 = da1 * tanh_derivative(z1)

        dW1 = X_batch.T @ dz1
        db1 = np.sum(dz1, axis=0, keepdims=True)
```

```
        # Gradient-descent update
        W1 -= learning_rate * dW1
        b1 -= learning_rate * db1
        W2 -= learning_rate * dW2
        b2 -= learning_rate * db2

    # Measure full-dataset loss after each epoch
    p_all, _ = forward(X)
    loss = binary_cross_entropy(y, p_all)
    loss_history.append(loss)

    if epoch % 1000 == 0:
        print(f"Epoch {epoch:5d} | Loss = {loss:.6f}")

# 6. Final evaluation
p_all, _ = forward(X)
predictions = (p_all >= 0.5).astype(int)

print("\nFINAL XOR PREDICTIONS")
for i in range(n):
    print(
        f"Input={X[i].astype(int)} "
        f"Probability={p_all[i, 0]:.4f} "
        f"Prediction={predictions[i, 0]} "
        f"Target={int(y[i, 0])}"
    )

accuracy = np.mean(predictions == y)
print(f"\nAccuracy: {accuracy:.2%}")

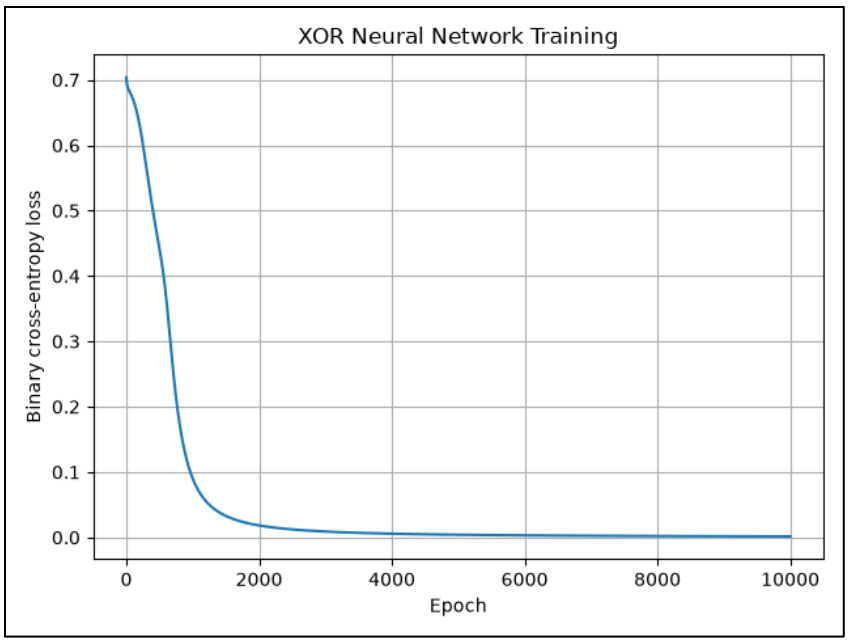
# 7. Plot the training loss
import matplotlib.pyplot as plt

plt.plot(loss_history)
plt.xlabel("Epoch")
plt.ylabel("Binary cross-entropy loss")
plt.title("XOR Neural Network Training")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\Neural Networks> py 09_training_loop.py
Epoch     0 | Loss = 0.703575
Epoch  1000 | Loss = 0.090060
Epoch  2000 | Loss = 0.018330
Epoch  3000 | Loss = 0.009003
Epoch  4000 | Loss = 0.005720
Epoch  5000 | Loss = 0.004108
Epoch  6000 | Loss = 0.003168
Epoch  7000 | Loss = 0.002561
Epoch  8000 | Loss = 0.002139
Epoch  9000 | Loss = 0.001830

FINAL XOR PREDICTIONS
Input=[0 0] Probability=0.0005 Prediction=0 Target=0
Input=[0 1] Probability=0.9981 Prediction=1 Target=1
Input=[1 0] Probability=0.9983 Prediction=1 Target=1
Input=[1 1] Probability=0.0023 Prediction=0 Target=0

Accuracy: 100.00%
```



14. Weight initialization

Suppose a hidden layer has two neurons, and we initialize their weights and biases identically:

$$W = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}, \qquad b = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Both neurons receive the same input, compute the same weighted sum, and produce the same activation.
During backpropagation, they also receive identical gradient signals. Their parameters therefore remain identical after updates.
This is called the symmetry problem: the neurons do not naturally develop distinct roles.
A network with multiple hidden neurons needs a way to break this symmetry. Random initialization is one common approach.
A useful distinction: initializing weights to zero is problematic for a hidden layer whose neurons have identical connections and activations. Some parameters, such as biases, are commonly initialized to zero without causing this particular symmetry problem when the weights are initialized differently.

Random initialization: Instead of setting every weight to zero, we can sample different initial weights from a random distribution.

```
import numpy as np

rng = np.random.default_rng(42)

W = rng.normal(0, 0.1, size=(2, 3))
print(W)
```

generates six random values from a normal distribution with:

- Mean $\mu = 0$
- Standard deviation $\sigma = 0.1$
- Shape 2×3

10_initialization.py

```
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

# Network layer dimensions
fan_in = 100
fan_out = 50
```

```
# 1. Small random initialization
W_small = rng.normal(
    0, 0.01, size=(fan_in, fan_out)
)

# 2. Xavier normal initialization
xavier_std = np.sqrt(2 / (fan_in + fan_out))
W_xavier = rng.normal(
    0, xavier_std, size=(fan_in, fan_out)
)

# 3. He normal initialization
he_std = np.sqrt(2 / fan_in)
W_he = rng.normal(
    0, he_std, size=(fan_in, fan_out)
)

# Print theoretical and observed standard deviations
print("INITIALIZATION COMPARISON")
print("-" * 45)

for name, weights, expected_std in [
    ("Small random", W_small, 0.01),
    ("Xavier normal", W_xavier, xavier_std),
    ("He normal", W_he, he_std)
]:
    print(f"\n{name}")
    print(f"Expected standard deviation: {expected_std:.5f}")
    print(f"Observed standard deviation: {np.std(weights):.5f}")
    print(f"Minimum weight: {np.min(weights):.5f}")
    print(f"Maximum weight: {np.max(weights):.5f}")

# Plot the weight distributions
plt.figure(figsize=(9, 5))

plt.hist(
    W_small.ravel(), bins=50,
    alpha=0.6, label="Small random"
)
plt.hist(
    W_xavier.ravel(), bins=50,
    alpha=0.6, label="Xavier normal"
)
plt.hist(
    W_he.ravel(), bins=50,
    alpha=0.6, label="He normal"
)

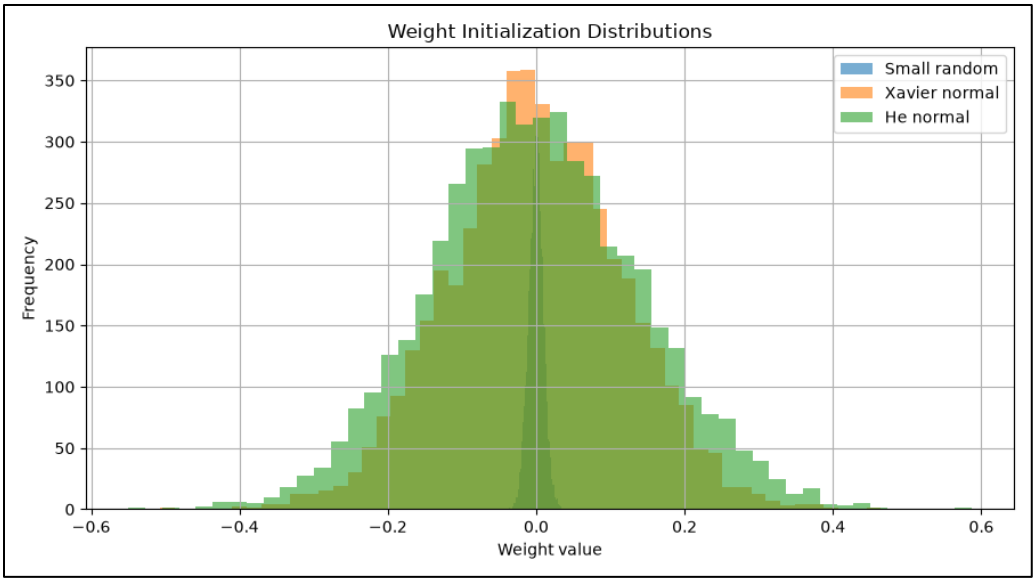
plt.xlabel("Weight value")
plt.ylabel("Frequency")
plt.title("Weight Initialization Distributions")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\Neural Networks> py 10_initialization.py
INITIALIZATION COMPARISON
------------------------------------------------

Small random
Expected standard deviation: 0.01000
Observed standard deviation: 0.00999
Minimum weight: -0.03648
Maximum weight: 0.03454

Xavier normal
Expected standard deviation: 0.11547
Observed standard deviation: 0.11698
Minimum weight: -0.50681
Maximum weight: 0.46486

He normal
Expected standard deviation: 0.14142
Observed standard deviation: 0.14134
Minimum weight: -0.54982
Maximum weight: 0.58707
```



11_gradient_clipping.py

```
import numpy as np

gradient = np.array([30.0, 40.0])

max_norm = 5.0
norm = np.linalg.norm(gradient)

if norm > max_norm:
    gradient = gradient * (max_norm / norm)

print("Clipped gradient:", gradient)
print("New norm:", np.linalg.norm(gradient))
```

```
PS C:\Users\Lenovo\ML Journey\Neural Networks> py 11_gradient_clipping.py
Clipped gradient: [3. 4.]
New norm: 5.0
```

15. Generalization

Imagine training a neural network to recognize faulty digital circuits. We provide examples of faulty and non-faulty circuits. The network learns from these examples, but your ultimate goal is not to classify only those exact examples correctly. We want it to classify new, unseen circuits correctly too. That ability is called generalization. A network can achieve very low training loss and still perform poorly on new examples. This is one reason training accuracy alone is not enough to judge a model.

a. Underfitting: The model is too limited, or has not trained sufficiently, to capture the underlying pattern. Typical symptom: high training error and high validation error.
b. Overfitting: The model fits training-specific details, including noise, and fails to generalize as well. Typical symptom: low training error but much higher validation error. A useful way to think about the distinction:
   - Underfitting: the model has not learned enough of the pattern.
   - Overfitting: the model has learned training-specific details that do not reliably transfer to new examples.
   - Good generalization: the model performs well on both training data and appropriately held-out data.

Training, validation, and test sets

Dataset	Purpose
Training set	Learn model parameters using gradients
Validation set	Compare models, tune hyperparameters, and decide when to stop
Test set	Evaluate the final selected model on held-out data

The validation set helps us make development decisions. The test set should not repeatedly influence those decisions; otherwise, it gradually becomes part of the model-selection process. For a small dataset, you might use a 70% / 15% / 15% split, but the right proportions depend on the dataset size and task.

16. Regularization

1) **L2 regularization:** One way to reduce overfitting is to add a penalty for large weights to the training objective.
   Without regularization, the objective is simply:

$$J_{\text{data}} = \frac{1}{N}\sum_{i=1}^{N} \ell_i$$

   With L2 regularization:

$$\boxed{J = J_{\text{data}} + \frac{\lambda}{2}\sum_{j} w_j^2}$$

   - J_{data} is the original training loss.
   - w_j represents a weight parameter.
   - λ controls the regularization strength.
   - The factor $1/2$ is included to simplify differentiation.

   The gradient becomes:

$$\frac{\partial J}{\partial w_j} = \frac{\partial J_{\text{data}}}{\partial w_j} + \lambda w_j$$

   So gradient descent updates a weight using:

$$w_j \leftarrow w_j - \eta\left(\frac{\partial J_{\text{data}}}{\partial w_j} + \lambda w_j\right)$$

The additional term discourages large weights. In neural-network training, L2 regularization is often called weight decay, although implementation details differ between coupled L2 penalties and decoupled weight-decay optimizers such as AdamW.

2) L1 regularization: L1 regularization adds the absolute values of the weights:

$$J = J_{\text{data}} + \lambda \sum_j |w_j|$$

Unlike L2, L1 can encourage some weights to become exactly zero, producing a sparse parameter representation. For a nonzero weight, the penalty's derivative is:

$$\frac{d|w|}{dw} = \begin{cases} 1, & w > 0 \\ -1, & w < 0 \end{cases}$$

At zero, the absolute-value function is not differentiable in the ordinary sense; optimization methods handle this using subgradients or specialized algorithms.

L1	L2
Penalizes absolute weight magnitude	Penalizes squared weight magnitude
Can encourage exact zeros	Typically shrinks weights toward zero
Useful when sparse solutions are desirable	Useful for discouraging large weights

17. Dropout

Dropout reduces over-reliance on particular hidden units by randomly deactivating some of them during training. The network must learn useful representations that remain effective even when some other units are absent.
Suppose a hidden layer has four activations: $a = [0.8 \quad 0.3 \quad 0.9 \quad 0.5]$
A particular dropout mask might be: $m = [1 \quad 0 \quad 1 \quad 0]$
The second and fourth units are temporarily deactivated.
With inverted dropout, the training activations are: $\tilde{a} = \frac{a \odot m}{1-p}$
where p is the dropout probability and $\odot$ means elementwise multiplication.
For $p = 0.5$: $\tilde{a} = [1.6 \quad 0 \quad 1.8 \quad 0]$
The surviving activations are scaled by $1/(1-p) = 2$. This keeps their expected magnitude aligned with the undropped activations.
During ordinary inference, dropout is disabled. Inverted dropout already applies the required scaling during training.

18. Early stopping

Training loss often continues to decrease as a network trains longer. Validation loss may decrease initially and then start increasing as the network begins to overfit. A common early-stopping procedure is:

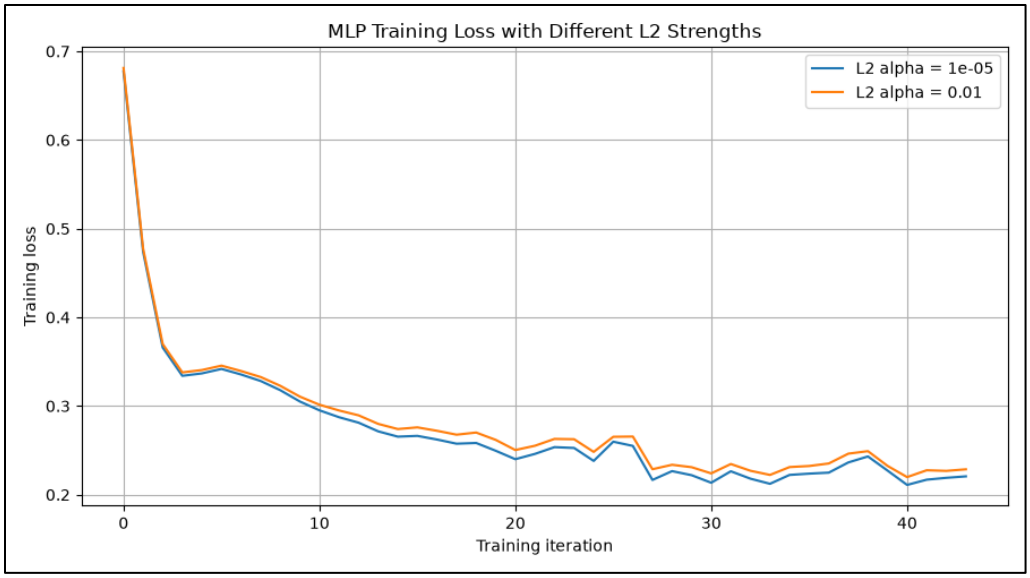
1. Train for one epoch.
2. Measure training and validation loss.
3. Save the model when validation loss improves.
4. Count consecutive epochs without meaningful improvement.
5. Stop after a chosen patience period and restore the best checkpoint.

This prevents the final model from simply being the last model trained.

```
PS C:\Users\Lenovo\ML Journey\Neural Networks> py 12_regularization.py

L2 regularization alpha = 1e-05
Training accuracy:   0.906
Validation accuracy: 0.887
Test accuracy:       0.919

L2 regularization alpha = 0.01
Training accuracy:   0.906
Validation accuracy: 0.881
Test accuracy:       0.919
```


MLP Training Loss with Different L2 Strengths
L2 alpha = 1e-05
L2 alpha = 0.01
Training loss
Training iteration
0.7
0.6
0.5
0.4
0.3
0.2
0
10
20
30
40

Convolutional Neural Networks (CNN)

CNNs are particularly useful for image data because they can learn spatial patterns such as edges, corners, textures, and larger structures.
Suppose we have a grayscale image of a semiconductor wafer with dimensions 256×256. It contains: $256 \times 256 = 65{,}536$ pixels. If we flatten this image into a vector, a fully connected neuron with one weight per pixel needs 65,536 weights, plus a bias. For an image with three color channels, the input would contain: $256 \times 256 \times 3 = 196{,}608$ input values.
A CNN uses a different idea: learn a small filter and slide it across the image. The same filter weights are reused at different locations, reducing the number of parameters and helping the model detect a pattern wherever it appears.

1. Representing an image as a tensor
A grayscale image can be represented as a matrix: $X \in \mathbb{R}^{H \times W}$, where H is image height and W is image width.
A color image usually has three channels—red, green, and blue: $X \in \mathbb{R}^{H \times W \times C}$, where $C = 3$ for RGB. A batch of images adds another dimension. In common deep-learning frameworks such as PyTorch, image batches typically use the layout: (N, C, H, W), where N is the batch size. Other libraries may use a different layout, so always check the expected tensor dimensions.

2. Convolution filter
A filter, also called a kernel, is a small matrix of weights. It examines a local region of an image and calculates a weighted sum. For example, consider this 3×3 filter:

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

This resembles a vertical-edge detector: it compares values on the left with values on the right.
A CNN normally learns its filter weights during training rather than relying on hand-designed filters like this one.
Example: suppose a grayscale image patch is: $X = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 1 & 2 \\ 0 & 1 & 3 \end{bmatrix}$ and the filter is: $K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$
Multiply corresponding entries and add:

$$\begin{aligned} s = {} & (1)(1) + (2)(0) + (0)(-1) \\ & +(3)(1) + (1)(0) + (2)(-1) \\ & +(0)(1) + (1)(0) + (3)(-1) \end{aligned}$$

Therefore:

$$s = 1 + 0 + 0 + 3 + 0 - 2 + 0 + 0 - 3 = \boxed{-1}$$

The filter produces one output value, -1, for this image patch.

3. Feature maps
The filter slides over different image regions. Each region produces one output value. Together, these values form a feature map. A CNN commonly learns many filters in a layer. One filter may respond to edges, another to a different orientation, and others to textures or more complex patterns. The exact patterns are learned from the training data.

4. Stride and Padding: Two settings control how convolution moves across an image.
   a. Stride is the number of pixels the filter moves at each step.
      - Stride 1: move one pixel at a time.
      - Stride 2: move two pixels at a time, usually producing a smaller output.
   b. Padding adds values around the image boundary, commonly zeros.
      - valid padding: no padding; the filter must fit entirely inside the input.
      - same padding: padding is chosen to preserve the spatial dimensions when stride is 1.

      For an input of size $H \times W$, a filter of size $K_H \times K_W$, padding P, and stride S, the output dimensions are:

$$H_{\text{out}} = \left\lfloor \frac{H + 2P - K_H}{S} \right\rfloor + 1$$
$$W_{\text{out}} = \left\lfloor \frac{W + 2P - K_W}{S} \right\rfloor + 1$$

Example-
Input: 28×28
Filter: 3×3
Stride: 1
Padding: 0

$$H_{\text{out}} = \frac{28 - 3}{1} + 1 = 26$$

The output feature map is 26×26.
With padding $P = 1$ and stride 1, the output becomes 28×28.

5. Multiple filters and channels

A real CNN does not usually use just one filter. If the input is an RGB image with three channels, one convolution filter spans all three channels. Each channel contributes to the output, and the results are summed together with a bias.
If a convolution layer has 16 filters, it produces 16 output feature maps.
For an input with C_{in} channels and C_{out} filters, the number of learned convolution weights is: $\boxed{K_H K_W C_{\text{in}} C_{\text{out}}}$
With one bias per output filter, the total number of parameters is: $\boxed{(K_H K_W C_{\text{in}} + 1) C_{\text{out}}}$
For example, a 3×3 convolution with 3 input channels and 16 output filters has: $(3 \cdot 3 \cdot 3 + 1) \cdot 16 =$ $\boxed{448 \text{ parameters}}$
That is far fewer than connecting every input pixel directly to every output feature in a large image.

6. ReLU and pooling

After convolution, CNNs often apply an activation function such as ReLU: $\text{ReLU}(z) = \max(0, z)$
For example: $\begin{bmatrix} -2 & 3 \\ 1 & -4 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 3 \\ 1 & 0 \end{bmatrix}$
ReLU replaces negative values with zero.

Max pooling: Pooling reduces the spatial dimensions of a feature map. A 2×2 max-pooling operation with stride 2 takes the maximum value from each non-overlapping 2×2 region.
For example:

$$\begin{bmatrix} 1 & 3 & 2 & 0 \\ 4 & 2 & 1 & 5 \\ 0 & 1 & 7 & 2 \\ 3 & 2 & 4 & 6 \end{bmatrix}$$

becomes:

$$\begin{bmatrix} 4 & 5 \\ 3 & 7 \end{bmatrix}$$

This reduces a 4×4 feature map to 2×2.
Pooling is one way to reduce computation and introduce some tolerance to small spatial shifts. Modern CNNs sometimes use strided convolutions or other downsampling methods instead.
A typical CNN feature-extraction pipeline is:

$$\boxed{\text{Image} \rightarrow \text{Convolution} \rightarrow \text{ReLU} \rightarrow \text{Downsampling}}$$

This sequence can be repeated through multiple layers. A classification network then uses the extracted features to predict a class.

13_cnn_basics.py

```
import numpy as np
```

```
import matplotlib.pyplot as plt

# 1. A small grayscale image
image = np.array([
    [1, 2, 0, 1, 3],
    [3, 1, 2, 0, 1],
    [0, 1, 3, 2, 0],
    [2, 4, 1, 3, 1],
    [1, 0, 2, 4, 3]
], dtype=float)

# 2. A 3x3 vertical-edge filter
kernel = np.array([
    [1, 0, -1],
    [1, 0, -1],
    [1, 0, -1]
], dtype=float)


# 3. Implement valid cross-correlation
def convolve_valid(image, kernel, stride=1):
    height, width = image.shape
    kh, kw = kernel.shape

    out_h = (height - kh) // stride + 1
    out_w = (width - kw) // stride + 1

    output = np.zeros((out_h, out_w))

    for i in range(out_h):
        for j in range(out_w):
            patch = image[
                i * stride:i * stride + kh,
                j * stride:j * stride + kw
            ]

            output[i, j] = np.sum(patch * kernel)

    return output


# 4. Convolution
feature_map = convolve_valid(image, kernel)

# 5. ReLU activation
relu_map = np.maximum(0, feature_map)


# 6. 2x2 max pooling with stride 2
def max_pool_2x2(matrix):
    height, width = matrix.shape

    out_h = height // 2
    out_w = width // 2

    pooled = np.zeros((out_h, out_w))

    for i in range(out_h):
        for j in range(out_w):
            patch = matrix[
                2*i:2*i+2,
```

```
                2*j:2*j+2
            ]

            pooled[i, j] = np.max(patch)

    return pooled


pooled_map = max_pool_2x2(relu_map)

# 7. Display numerical results
print("INPUT IMAGE:")
print(image)

print("\nKERNEL:")
print(kernel)

print("\nFEATURE MAP:")
print(feature_map)

print("\nAFTER RELU:")
print(relu_map)

print("\nAFTER MAX POOLING:")
print(pooled_map)

# 8. Plot all stages
fig, axes = plt.subplots(1, 4, figsize=(13, 3.5))

stages = [
    (image, "Input Image"),
    (feature_map, "Feature Map"),
    (relu_map, "After ReLU"),
    (pooled_map, "After Max Pooling")
]

for ax, (matrix, title) in zip(axes, stages):
    ax.imshow(matrix, cmap="gray", interpolation="nearest")
    ax.set_title(title)
    ax.set_xticks(range(matrix.shape[1]))
    ax.set_yticks(range(matrix.shape[0]))

    for i in range(matrix.shape[0]):
        for j in range(matrix.shape[1]):
            ax.text(
                j, i, f"{matrix[i, j]:.0f}",
                ha="center", va="center", color="red"
            )

plt.tight_layout()
plt.show()
```

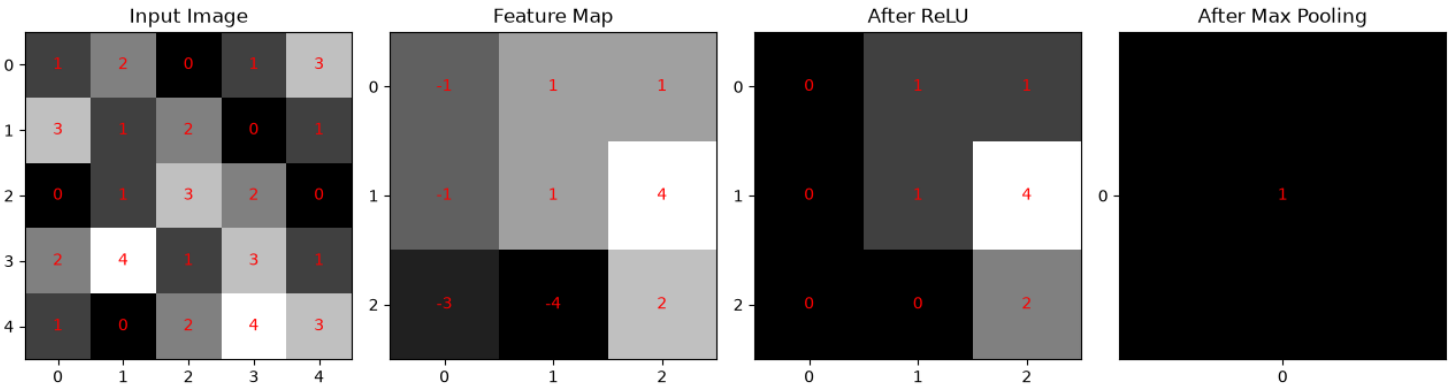
```
PS C:\Users\Lenovo\ML Journey\Neural Networks> py 13_cnn_basics.py
INPUT IMAGE:
[[1. 2. 0. 1. 3.]
 [3. 1. 2. 0. 1.]
 [0. 1. 3. 2. 0.]
 [2. 4. 1. 3. 1.]
 [1. 0. 2. 4. 3.]]

KERNEL:
[[ 1.  0. -1.]
 [ 1.  0. -1.]
 [ 1.  0. -1.]]

FEATURE MAP:
[[-1.  1.  1.]
 [-1.  1.  4.]
 [-3. -4.  2.]]

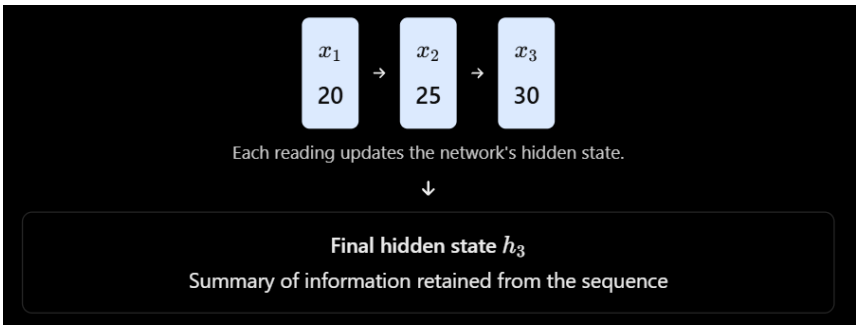
AFTER RELU:
[[0. 1. 1.]
 [0. 1. 4.]
 [0. 0. 2.]]

AFTER MAX POOLING:
[[1.]]
```



Recurrent Neural Networks (RNNs)

A basic feed-forward neural network receives a fixed-size input and computes an output. An RNN processes a sequence one step at a time, carrying information from earlier steps through a hidden state.
Suppose our input is a sequence of three temperature readings: $x_1 = 20,\ x_2 = 25,\ x_3 = 30$
The RNN processes them in order:



Unlike a network that treats each reading independently, the RNN can use its previous hidden state when processing the next reading.

1. The RNN cell

At time step t, an RNN receives two things:

- x_t: the current input.
- h_{t-1}: the previous hidden state.

It produces a new hidden state h_t.
The standard RNN equations are:

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$
$$y_t = W_{hy}h_t + b_y$$

Symbol	Meaning
x_t	Input at the current time step
h_{t-1}	Previous hidden state
h_t	Updated hidden state
W_{xh}	Input-to-hidden weights
W_{hh}	Hidden-to-hidden weights
W_{hy}	Hidden-to-output weights
b_h, b_y	Bias terms
tanh	Nonlinear activation function
y_t	Output at the current time step

The same weight matrices are reused at every time step. This is called parameter sharing.

2. Numerical example

(simplest possible RNN) one input, one hidden state, and one output, where all quantities are scalars rather than vectors or matrices.
Assume: $x_1 = 1, \quad x_2 = 2, \quad x_3 = 3$
Initial hidden state: $h_0 = 0$
Choose the parameters: $W_{xh} = 0.5, \quad W_{hh} = 0.2, \quad b_h = 0$
For now, we will ignore the output layer and calculate only the hidden states.
Our equation becomes: $h_t = \tanh(0.5x_t + 0.2h_{t-1})$
Time step 1: Input $x_1 = 1$, previous hidden state $h_0 = 0$.

$$\begin{aligned} h_1 &= \tanh(0.5(1) + 0.2(0))) \\ &= \tanh(0.5) \\ &\approx 0.4621 \end{aligned}$$

Time step 2: Input $x_2 = 2$, previous hidden state $h_1 = 0.4621$.

$$\begin{aligned} h_2 &= \tanh(0.5(2) + 0.2(0.4621))) \\ &= \tanh(1.0924) \\ &\approx 0.7977 \end{aligned}$$

Notice that h_2 depends on both the current input x_2 and the information carried by h_1.

Time step 3: Input $x_3 = 3$, previous hidden state $h_2 = 0.7977$.

$$\begin{aligned} h_3 &= \tanh(0.5(3) + 0.2(0.7977))) \\ &= \tanh(1.6595) \\ &\approx 0.9301 \end{aligned}$$

Numerical results

Time step	Input	Previous state	New state
1	1	0	0.4621
2	2	0.4621	0.7977
3	3	0.7977	0.9301

The hidden state increases as the inputs increase in this example. This is a consequence of our chosen weights and inputs, not a rule that every RNN must follow.

14_simple_rnn.py

```
import numpy as np

# Input sequence
sequence = [1.0, 2.0, 3.0]

# RNN parameters
W_xh = 0.5
W_hh = 0.2
b_h = 0.0

# Initial hidden state
h = 0.0

# Process each input in order
for t, x in enumerate(sequence, start=1):

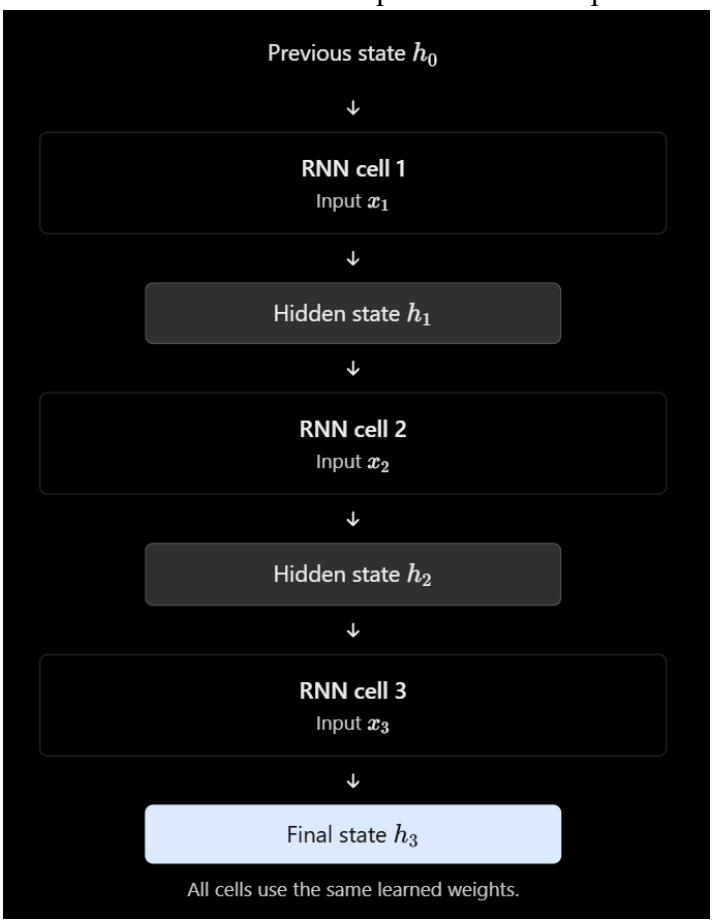
    # Calculate the new hidden state
    z = W_xh * x + W_hh * h + b_h
    h = np.tanh(z)

    print(f"Time step {t}")
    print(f"Input: {x:.1f}")
    print(f"Weighted sum: {z:.4f}")
    print(f"Hidden state: {h:.4f}")
    print("-" * 25)
```

```
PS C:\Users\Lenovo\ML Journey\Neural Networks> py 14_simple_rnn.py
Time step 1
Input: 1.0
Weighted sum: 0.5000
Hidden state: 0.4621
-------------------------
Time step 2
Input: 2.0
Weighted sum: 1.0924
Hidden state: 0.7978
-------------------------
Time step 3
Input: 3.0
Weighted sum: 1.6596
Hidden state: 0.9302
-------------------------
```

3. unrolling through time

A recurrent network can be visualized as the same computational cell repeated across the input sequence.



4. Different RNN input-output arrangements RNNs can produce outputs in different ways.

A. Many-to-one: Many inputs, one final output
   Example: classify a sequence of sensor readings as normal or faulty.
   Input: $x_1, x_2, x_3 \rightarrow$ Output: one class
B. Many-to-many: Many inputs, outputs at multiple time steps
   Example: label each time step in a sensor sequence.
   Inputs: $x_1, x_2, x_3 \rightarrow$ Outputs: y_1, y_2, y_3
C. One-to-many: One input generates a sequence of outputs.
   For example, a model may generate a sequence conditioned on a single starting representation.

5. Backpropagation Through Time (BPTT)

So far, we manually chose the weights. In a real neural network, the weights must be learned from training data. Suppose the RNN predicts a target value $\hat{y}$, and the correct answer is y. We can calculate the squared-error loss:

$$L = \frac{1}{2}(y - \hat{y})^2$$

Training aims to adjust the weights to reduce this loss.

The challenge is that the final output depends on earlier hidden states, which depend on earlier inputs and the same recurrent weights.

1. The training process is Forward pass through the sequence
   ↓
2. Calculate prediction error
   ↓
3. Propagate gradients backward through time
   ↓

4. Update shared weights

↓

Repeat over training sequences

This is Backpropagation Through Time (BPTT). It applies the chain rule across the unrolled sequence to calculate how the loss depends on the shared weights.

During backpropagation, gradients pass through many recurrent steps. Repeated multiplication by derivatives and weights can cause gradients to:

- Vanishing gradients: become extremely small, making it difficult to learn long-term dependencies.
- Exploding gradients: become extremely large, making training unstable.

Two important solutions are:

- LSTM (Long Short-Term Memory): uses gates and a cell state to help preserve useful information over longer sequences.
- GRU (Gated Recurrent Unit): uses a simpler gating mechanism to control how information is retained and updated.

LSTMs and GRUs are recurrent architectures designed to improve information flow; they do not guarantee that every long-term dependency will be learned.